

APLICACIONES MOVILES

LABORATORIO 3A

La misma app, tres veces: el modelo tradicional

Módulo 2 — Desarrollo nativo · Guía autónoma · Tiempo estimado: 90 minutos · Primera parte de tres (3A → 3B → 3C)

El problema de hoy: vas a construir la misma aplicación tres veces, con las tres formas de desarrollar que conviven en el Android real. Esta primera vez la construís como se construyó durante una década: el modelo tradicional puro — con toda su ceremonia y con su talón de Aquiles a la vista. Al final de la trilogía vas a poder responder con fundamento por qué la industria cambió de forma dos veces.

Qué necesitás antes de empezar: los Laboratorios 1 y 2 completos — este da por sabido crear proyectos, correr en emulador/teléfono, leer Logcat y navegar con Intents. Y algo nuevo: **elegí tus 8 aplicaciones favoritas** (las que más usás o más admirás) y anotá de cada una el nombre, la categoría (mensajería, juegos, finanzas...) y una línea de por qué te gusta. Son tus datos de trabajo en las tres partes.

Teoría que acompaña: la estructura de la interfaz y la navegación, y —sobre todo— el bloque de la evolución de las formas de desarrollar: esta trilogía es ese bloque, construido con tus manos.

La ficha comparativa (importante): al final de cada parte vas a responder las mismas seis preguntas por escrito. Al terminar 3C vas a tener una tabla de tres columnas que es *tuya* — tu comparación, con tu experiencia — y que vas a poder citar en la defensa del trabajo práctico. Guardala desde hoy.

El destino

Al terminar esta parte vas a tener **AppTeca**: tu catálogo personal de aplicaciones favoritas — una lista con tus 8 apps, cada una tocable hacia su pantalla de detalle, con estrella para marcar favoritas, un buscador que filtra en vivo y un modo "solo favoritas". Construida 100% en el modelo tradicional: layouts XML, Activities, un RecyclerView con su adaptador, y el estado viviendo donde el modelo tradicional lo pone — en las variables de la pantalla. La última etapa va a girar el dispositivo, y lo que pase ahí es el punto de partida de la parte 3B.

Etapas 1 — El proyecto y tus datos

Todo catálogo empieza por sus datos. Esta etapa crea el proyecto y modela tu colección — sin una sola pantalla todavía.

1.1 — Creá el proyecto. Como en el Laboratorio 1 (esta vez sin guía fina — ya sabés): **New Project → Empty Views Activity** (la de Views, no la primera de la galería) · Name: `AppTeca` · Language: Kotlin · Minimum SDK: API 26. Esperá la sincronización de Gradle.

✓ *Verificación: proyecto abierto, sin errores, con `activity_main.xml` bajo `res/layout` (la prueba de que elegiste la plantilla correcta).*

1.2 — El modelo. Creá un archivo nuevo para el modelo: click derecho sobre el paquete → **New → Kotlin Class/File** → tipo `File`, nombre `App`. Adentro:

```
package com.example.appteca

data class App(
    val id: Int,
    val nombre: String,
    val categoria: String,
```

```

    val descripcion: String,
    var esFavorita: Boolean = false
)

```

Dos decisiones de diseño para leer antes de seguir: el `id` existe porque las pantallas van a necesitar referirse a una app sin pasarse el objeto entero (te va a sonar del Lab 2: los extras de un Intent viajan mejor livianos). Y `esFavorita` es `var` —mutable— mientras todo lo demás es `val`: en el modelo tradicional el estado se muta en el lugar, y esta variable es exactamente el estado que este lab va a poner en peligro. En la parte 3C vas a ver que esta decisión se invierte — anotalo mentalmente.

1.3 — Tus datos. Nuevo archivo Kotlin, tipo File, nombre `Catalogo`:

```

package com.example.appteca

object Catalogo {
    val apps = mutableListOf(
        App(1, "WhatsApp", "Mensajería", "La uso para todo, hasta para mandarme cosas a mí mismo"),
        App(2, "Spotify", "Música", "Las playlists me organizan el día"),
        // ... reemplazá estas y completá con TUS 8 apps ...
    )
}

```

Cargá tus 8 con tus descripciones reales — en 3B y 3C vas a reusar este mismo archivo, así que hacelo bien ahora. El `object` es un singleton de Kotlin: una única instancia global — nuestro "repositorio" de juguete, suficiente mientras los datos vivan en memoria (la persistencia real es tema de la teoría más adelante).

✓ Verificación: los dos archivos compilan (Build → Make Project, o el martillito) sin errores, y tu catálogo tiene 8 apps con id únicos.

Problemas frecuentes de la Etapa 1

- **"Expecting a top level declaration"** o similar en `App.kt`: elegiste "Class" en vez de "File" al crear y quedó una clase envolvente duplicada — borra todo el contenido del archivo y pegá el código tal cual.
- **Los tildes o emojis de tus descripciones rompen algo**: no deberían — Kotlin es UTF-8. Si ves caracteres raros al ejecutar, el problema va a ser de la fuente del emulador, no de tu código. Seguí.

Etapa 2 — La lista: RecyclerView y su ceremonia

Toda app de catálogo es, antes que nada, una lista. En el modelo tradicional, la lista tiene su componente estrella — el RecyclerView — y su precio de entrada: la ceremonia del adaptador. Esta etapa paga ese precio completo, contándolo.

2.1 — El layout de la pantalla. Reemplazá el contenido de `activity_main.xml`:

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">

```

```

<androidx.recyclerview.widget.RecyclerView
    android:id="@+id/rvApps"
    android:layout_width="match_parent"
    android:layout_height="match_parent" />
</LinearLayout>

```

¿Por qué **RecyclerView** y no una columna de vistas a mano? Por el presupuesto de recursos de la teoría: una lista de mil elementos con mil vistas infladas es memoria y batería tiradas — el RecyclerView crea solo las vistas que entran en pantalla y las **recicla** al scrollear: la vista que sale por arriba se rellena con el dato que entra por abajo. Con 8 apps no lo vas a notar; con las listas reales del TP, sí. Es el componente más usado del Android tradicional — y el más ceremonioso, como estás por comprobar.

2.2 — El layout del ítem. La lista necesita saber cómo se ve *cada fila*. Nuevo archivo: click derecho sobre `res/layout` → **New** → **Layout Resource File**, nombre `item_app`:

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:gravity="center_vertical"
    android:padding="16dp">

    <LinearLayout
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:orientation="vertical">

        <TextView
            android:id="@+id/tvNombre"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:textSize="18sp"
            android:textStyle="bold" />

        <TextView
            android:id="@+id/tvCategoria"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:textSize="14sp" />
    </LinearLayout>

    <TextView
        android:id="@+id/tvEstrella"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:textSize="24sp"
        android:padding="8dp" />
</LinearLayout>

```

✓ Verificación: la vista Design de `item_app.xml` muestra la fila (vacía de textos, con la estructura visible).

2.3 — El adaptador: la pieza que traduce datos en filas. Acá está el corazón de la ceremonia. El RecyclerView no sabe nada de tus apps: necesita un **adaptador** — el objeto que responde sus tres

preguntas: *¿cuántos ítems hay?, ¿cómo se crea una fila vacía?, ¿cómo se llena la fila N con el dato N?*
 Nuevo archivo Kotlin, tipo File, `AppAdapter`:

```
package com.example.appteca

import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.TextView
import androidx.recyclerview.widget.RecyclerView

class AppAdapter(
    private var items: List<App>
) : RecyclerView.Adapter<AppAdapter.AppViewHolder>() {

    // El ViewHolder: "sostiene" las vistas de UNA fila ya encontradas,
    // para no buscarlas (findViewById) en cada scroll.
    class AppViewHolder(v: View) : RecyclerView.ViewHolder(v) {
        val tvNombre: TextView = v.findViewById(R.id.tvNombre)
        val tvCategoria: TextView = v.findViewById(R.id.tvCategoria)
        val tvEstrella: TextView = v.findViewById(R.id.tvEstrella)
    }

    // Pregunta 2: ¿cómo se crea una fila vacía? (se llama pocas veces: recicla)
    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int):
    AppViewHolder {
        val v = LayoutInflater.from(parent.context)
            .inflate(R.layout.item_app, parent, false)
        return AppViewHolder(v)
    }

    // Pregunta 3: ¿cómo se llena la fila con el dato de la posición?
    // (se llama TODO el tiempo: cada reciclado pasa por acá)
    override fun onBindViewHolder(holder: AppViewHolder, position: Int) {
        val app = items[position]
        holder.tvNombre.text = app.nombre
        holder.tvCategoria.text = app.categoria
        holder.tvEstrella.text = if (app.esFavorita) "★" else "☆"
    }

    // Pregunta 1: ¿cuántos hay?
    override fun getItemCount() = items.size
}
```

Leelo con el ojo en la estructura, no en la sintaxis: reconocí el gesto tradicional del Laboratorio 1 — encontrar vistas por id, mutarlas— ahora industrializado para listas. Y hacé la cuenta que la ficha comparativa te va a pedir: para mostrar una lista hicieron falta **cuatro piezas** — el layout del ítem, el ViewHolder, el adaptador con sus tres métodos, y el enganche que viene ahora.

2.4 — El enganche. En `MainActivity.kt`, el `onCreate` completo:

```
class MainActivity : AppCompatActivity() {

    private lateinit var adapter: AppAdapter

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        adapter = AppAdapter(Catalogo.apps)
        val rv = findViewById<RecyclerView>(R.id.rvApps)
    }
}
```

```

        rv.layoutManager = LinearLayoutManager(this)    // "en columna, de arriba
a abajo"
        rv.adapter = adapter
    }
}

```

(Imports: `androidx.recyclerview.widget.RecyclerView` y `androidx.recyclerview.widget.LinearLayoutManager` — **Alt+Enter** como siempre. El `lateinit` declara "la voy a inicializar antes de usarla, prometido" — la alternativa de Kotlin a un nulo provisorio.) Ejecutá.

✓ *Verificación: tu lista en pantalla — tus 8 apps, con nombre, categoría y la estrella vacía ☆ en cada fila. Scrolleá; tocá (todavía no pasa nada — eso es la próxima etapa).*

🔧 Problemas frecuentes de la Etapa 2

- **"Unresolved reference: RecyclerView"** en el XML o en Kotlin: la dependencia falta en proyectos viejos, pero la plantilla actual la trae vía Material. Si te pasa: agregá `implementation("androidx.recyclerview:recyclerview:1.3.2")` al bloque `dependencies` de `build.gradle.kts` (`Module :app`) y sincronizá [△ VERIFICAR: versión vigente].
- **La lista sale vacía:** el 90% de las veces es el `layoutManager` olvidado — sin él, el `RecyclerView` no sabe ni cómo acomodar las filas y no muestra nada, *sin error*. El otro 10%: tu `Catalogo.apps` quedó vacío.
- **"lateinit property adapter has not been initialized"**: usaste `adapter` antes de la línea que lo crea — revisá el orden dentro de `onCreate`.

Etapa 3 — El detalle: navegar con equipaje liviano

Cada app de tu catálogo merece su pantalla. La navegación es la del Laboratorio 2 — con una decisión de diseño nueva: qué viaja en el Intent.

3.1 — La pantalla de detalle. Creá `DetalleActivity` (**New → Activity → Empty Views Activity** — se registra sola en el manifiesto, ya sabés). Su layout, `activity_detalle.xml`:

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="24dp">

    <TextView
        android:id="@+id/tvDetNombre"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:textSize="28sp"
        android:textStyle="bold" />

    <TextView
        android:id="@+id/tvDetCategoria"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:textSize="16sp"
        android:layout_marginTop="4dp" />

```

```

<TextView
    android:id="@+id/tvDetDescripcion"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:textSize="16sp"
    android:layout_marginTop="16dp" />
</LinearLayout>

```

3.2 — El click en la fila: el adaptador avisa hacia arriba. El adaptador no debe saber de Intents ni de pantallas — su trabajo es la lista. Lo correcto es que *avise* que tocaron una fila y que la Activity decida qué hacer: una **lambda como parámetro del constructor** (el patrón observador en versión de bolsillo). Modificá la firma del adaptador y agregó el listener en el bind:

```

class AppAdapter(
    private var items: List<App>,
    private val onAppClick: (App) -> Unit
) : RecyclerView.Adapter<AppAdapter.AppViewHolder>() {

```

```

    override fun onBindViewHolder(holder: AppViewHolder, position: Int) {
        val app = items[position]
        holder.tvNombre.text = app.nombre
        holder.tvCategoria.text = app.categoria
        holder.tvEstrella.text = if (app.esFavorita) "★" else "☆"
        holder.itemView.setOnClickListener { onAppClick(app) }
    }

```

3.3 — La Activity decide: viaja el id, no el objeto. En `MainActivity`, la creación del adaptador cambia a:

```

adapter = AppAdapter(Catalogo.apps) { app ->
    val intent = Intent(this, DetalleActivity::class.java)
    intent.putExtra("appId", app.id)
    startActivity(intent)
}

```

La decisión de diseño para leer: **viaja el id**, no la app entera. Se puede serializar el objeto completo en el extra, pero es el hábito que después duele — los extras son para *referencias livianas*; el destino busca el dato completo en la fuente compartida. Es la misma lógica del contrato del módulo web: por el canal viajan identificadores y datos mínimos; la verdad vive en el repositorio.

3.4 — El destino busca su dato. En `DetalleActivity`:

```

class DetalleActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_detalle)

        val appId = intent.getIntExtra("appId", -1)
        val app = Catalogo.apps.find { it.id == appId }

        if (app == null) { finish(); return } // el plan B: sin dato válido,
        esta pantalla no tiene sentido

        findViewById<TextView>(R.id.tvDetNombre).text = app.nombre
        findViewById<TextView>(R.id.tvDetCategoria).text = app.categoria
    }
}

```

```

        findViewById<TextView>(R.id.tvDetDescripcion).text = app.descripcion
    }
}

```

Ejecutá.

✓ *Verificación: tocás cualquier fila → su detalle completo; atrás → la lista, como la dejaste. Fijate el plan B del `null`: un id que no existe cierra la pantalla en vez de crashear — el hábito de la nulabilidad, aplicado a la navegación.*

Problemas frecuentes de la Etapa 3

- **El detalle sale vacío o se cierra solo:** la clave del extra no coincide ("`appId`" exacto en ambos lados) o tus ids del Catálogo no son únicos — el `find` devuelve el primero que matchea o `null`.
- **El click no hace nada:** el `setOnClickListener` va sobre `holder.itemView` (la fila entera), adentro de `onBindViewHolder` — verificá que no lo pusiste en el `ViewHolder` por error.
- **"None of the following candidates is applicable"** al crear el adaptador: te falta actualizar la llamada en `MainActivity` con la lambda nueva — la firma del constructor cambió.

Etapa 4 — Favoritos y filtros: el estado, y vos como sincronizador

Hasta acá la app muestra datos quietos. Ahora entra el estado — lo que cambia mientras se usa — y con él, el trabajo que define al modelo tradicional: mantener a mano la pantalla y los datos diciendo lo mismo.

4.1 — La estrella que marca. En el adaptador, la estrella pasa a responder al toque. Suma un segundo parámetro-lambda al constructor y el listener en el bind:

```

class AppAdapter(
    private var items: List<App>,
    private val onAppClick: (App) -> Unit,
    private val onFavoritoClick: (App) -> Unit
) : RecyclerView.Adapter<AppAdapter.AppViewHolder>() {

```

```

        holder.tvEstrella.setOnClickListener { onFavoritoClick(app) }

```

Y en `MainActivity`, la decisión de qué hacer con el aviso — mutar el dato y avisarle a la lista que se redibuje:

```

        adapter = AppAdapter(Catalogo.apps,
            onAppClick = { app ->
                val intent = Intent(this, DetalleActivity::class.java)
                intent.putExtra("appId", app.id)
                startActivity(intent)
            },
            onFavoritoClick = { app ->
                app.esFavorita = !app.esFavorita
                adapter.notifyDataSetChanged()
            })

```

Ese `notifyDataSetChanged()` merece dos oraciones: es la confesión del modelo — **el dato cambió y la vista no se enteró sola**; alguien (vos) tiene que ordenar el redibujado. Y es el martillo grande:

redibuja *todo* cuando cambió *uno* (existe una herramienta fina para redibujar solo lo que cambió — DiffUtil — que queda como desafío; para 8 filas, el martillo alcanza).

✓ Verificación: tocás ☆ → ★, tocás de nuevo → ☆. El dato y la vista, sincronizados — porque vos los sincronizaste.

4.2 — La primera grieta: el favorito desde el detalle. Agregale al detalle su propio botón de favorito. En `activity_detalle.xml`, al final:

```
<Button
    android:id="@+id/btnFavorito"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="24dp" />
```

Y en `DetalleActivity`, después de llenar los textos:

```
val btn = findViewById<Button>(R.id.btnFavorito)
fun pintar() { btn.text = if (app.esFavorita) "★ Quitar de favoritas"
else "☆ Marcar favorita" }
pintar()
btn.setOnClickListener { app.esFavorita = !app.esFavorita; pintar() }
```

Ahora el experimento — hazelo exactamente así: en la lista, entrá al detalle de una app **sin estrella**, marca favorita desde el botón, y **volvé atrás**. Mirá su fila en la lista.

✓ Verificación: la fila muestra ☆ — está desactualizada. El dato dice favorita (volvé a entrar al detalle: el botón lo confirma); la lista dice que no. ¿Por qué? Porque el dato cambió en otra pantalla, y nadie le avisó al adaptador de esta. La vista tradicional no observa los datos: muestra la última foto que alguien le ordenó sacar.

4.3 — El parche del sincronizador. El arreglo tradicional es tan simple como sintomático — en `MainActivity`:

```
override fun onResume() {
    super.onResume()
    adapter.notifyDataSetChanged() // "por si algo cambió mientras no
miraba"
}
```

Funciona (verificalo: repetí el experimento). Y leé lo que acabás de escribir: un redibujado total *por las dudas*, cada vez que la pantalla vuelve. En una app de dos pantallas es un parche; en una de veinte, es un campo minado — cada pantalla recordando avisarle a cada otra. Anotá la sensación: es la respuesta a una pregunta de la ficha.

4.4 — Buscador y "Solo favoritas": los filtros se diseñan juntos. El catálogo gana sus dos filtros de una vez: la búsqueda por texto y el modo "solo favoritas" — y antes de escribir una línea, la decisión de diseño que los gobierna: **cuando una pantalla tiene más de un filtro, la lista visible no se arma en cada listener — se recalcula entera en una única función, desde el estado actual**. Cada listener se limita a cambiar su pedacito de estado y llamar a esa función. Sin esa disciplina, cada listener recalcula "su" parte, los caminos se multiplican y las combinaciones se desincronizan.

Primero, las dos vistas nuevas en `activity_main.xml`, antes del `RecyclerView`:

```
<EditText
    android:id="@+id/etBuscar"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="@string/hint_buscar"
    android:inputType="text"
    android:padding="16dp" />

<Button
    android:id="@+id/btnSoloFav"
    android:layout_width="match_parent"
    android:layout_height="wrap_content" />
```

(String: `<string name="hint_buscar">Buscar por nombre o categoría...</string>`.) El adaptador necesita poder cambiar de lista — agregale este método:

```
fun actualizarLista(nueva: List<App>) {
    items = nueva
    notifyDataSetChanged()
}
```

Y ahora `MainActivity` **completa, como queda terminada** — lee la entera antes de reemplazar la tuya, porque la estructura es la lección:

```
class MainActivity : AppCompatActivity() {

    private lateinit var adapter: AppAdapter

    // El estado de la pantalla que NO vive en una vista:
    private var soloFavoritas = false

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        adapter = AppAdapter(Catalogo.apps,
            onAppClick = { app ->
                val intent = Intent(this, DetalleActivity::class.java)
                intent.putExtra("appId", app.id)
                startActivity(intent)
            },
            onFavoritoClick = { app ->
                app.esFavorita = !app.esFavorita
                aplicarFiltros() // cambió el estado → recalcular
            })

        val rv = findViewById<RecyclerView>(R.id.rvApps)
        rv.layoutManager = LinearLayoutManager(this)
        rv.adapter = adapter

        findViewById<EditText>(R.id.etBuscar).addTextChangedListener {
            aplicarFiltros() // cambió el estado → recalcular
        }

        findViewById<Button>(R.id.btnSoloFav).setOnClickListener {
            soloFavoritas = !soloFavoritas
            aplicarFiltros() // cambió el estado → recalcular
        }
    }
}
```

```

        aplicarFiltros() // estado inicial → primera foto
    }

    override fun onResume() {
        super.onResume()
        aplicarFiltros() // "por si algo cambió mientras no
miraba"
    }

    // LA función: de todo el estado actual, deriva la lista visible y el botón.
    private fun aplicarFiltros() {
        val q = findViewById<EditText>(R.id.etBuscar).text.toString().trim()
        var lista: List<App> = Catalogo.apps
        if (q.isNotEmpty()) lista = lista.filter {
            it.nombre.contains(q, true) || it.categoria.contains(q, true)
        }
        if (soloFavoritas) lista = lista.filter { it.esFavorita }
        adapter.actualizarLista(lista)
        findViewById<Button>(R.id.btnSoloFav).text =
            if (soloFavoritas) "★ Solo favoritas" else "☆ Todas"
    }
}

```

(Imports con Alt+Enter: `addTextChangedListener` es el de `androidx.core.widget`.) Fijate el patrón que quedó dibujado: **los listeners solo tocan estado y avisan; una función central deriva todo lo visible**. Hasta el `onResume` del parche de la etapa anterior ahora llama a la misma función. Es lo más cerca que el modelo tradicional puede estar de "la interfaz como función del estado" — con la diferencia decisiva de que acá la función *la llamas vos*, y olvidarte de llamarla en un solo camino es un bug.

✓ Verificación: escribís y la lista se achica en vivo; el botón alterna ☆ Todas / ★ Solo favoritas; los dos filtros se combinan (buscá con el modo activo); y si en modo favoritas desmarcás una estrella, la app desaparece de la vista al instante. Contá el estado que esta pantalla administra: el texto del buscador, el booleano del modo, los favoritos de cada app, y la lista derivada de los tres.

🔧 Problemas frecuentes de la Etapa 4

- **La estrella cambia recién al scrollear:** algún camino muta el dato sin pasar por `aplicarFiltros()` — buscá el listener que se olvidó de avisar.
- **El filtro rompe los favoritos** (marcás en la lista filtrada y se marca otra): estás usando la *posición* en vez del objeto en alguna lambda — las lambdas reciben `app`, no `position`; con la lista filtrada, la posición 0 ya no es la app 1.
- **`addTextChangedListener` no resuelve:** es el helper de `androidx.core.widget`, no el método clásico con tres callbacks — importá el correcto.
- **El botón arranca sin texto:** te faltó la llamada inicial a `aplicarFiltros()` al final de `onCreate`.

Etapa 5 — El giro fatal, versión app real — y tu ficha

El Laboratorio 2 te mostró el problema en un contador de juguete. Ahora mirá lo que le hace a una funcionalidad de verdad.

5.1 — El modo que se apaga solo. Marcá dos o tres favoritas, activá  **Solo favoritas** (la lista se achica), y ahora **rotá**.

✓ *Verificación: el modo se apagó solo — el botón volvió a "☆ Todas" y la lista muestra el catálogo completo. Nadie tocó nada: el usuario giró el teléfono y la app cambió de comportamiento, en silencio y sin error a la vista. La explicación es la del Laboratorio 2, ahora con consecuencias reales: `soloFavoritas` era una variable de la Activity que el giro destruyó; la Activity nueva nació con `false`, y `aplicarFiltros()` hizo su trabajo — con el estado equivocado. Es el peor tipo de bug: el que no crashea.*

5.2 — Y la muerte completa. Los favoritos, en cambio, sobrevivieron al giro — no por mérito tuyo: viven en `Catalogo`, el singleton, que es memoria *del proceso*, y el giro mata la Activity, no el proceso. Terminá el experimento con el arma del Laboratorio 2: favoritas marcadas, Home, y desde la Terminal `adb shell am kill com.example.appteca`. Volvé desde recientes.

✓ *Verificación: favoritos en blanco — el proceso murió y el singleton con él. El cuadro completo queda dibujado: cada estado de tu pantalla muere en un momento distinto — la variable de la Activity muere al girar; la memoria del proceso muere con el proceso — y en el modelo tradicional, saber qué vive dónde y reconstruirlo tras cada muerte es trabajo tuyo, artesanal, pantalla por pantalla.*

5.3 — La ficha comparativa: columna A. Abrió tu ficha y llená la primera columna respondiendo por escrito, con lo que acabás de vivir:

Pregunta	3A — Tradicional	3B	3C
¿Cuántas piezas hicieron falta para mostrar la lista?			
¿Dónde vive el estado de la pantalla (búsqueda, modo, favoritos)?			
¿Qué sobrevivió y qué murió al rotar? ¿Y al morir el proceso? ¿Por qué cada uno?			
¿Qué costaría agregar un campo nuevo al ítem (por ej., un puntaje)?			
¿Qué se podría probar de esta lógica sin abrir un emulador?			
¿Qué se sintió mejor y qué peor de esta forma de trabajar?			

✓ *Verificación: columna A completa. La pregunta 5 tiene una respuesta incómoda en 3A — pensala bien: ¿dónde está la lógica de los filtros? ¿Se puede ejecutar sin una Activity viva?*

Desafíos opcionales

Sin guía paso a paso — de eso se trata.

1. **El martillo fino.** Investiga **DiffUtil** (o `ListAdapter`, su envoltorio): la herramienta que calcula *qué* cambió entre dos listas y redibuja solo eso. Reemplazá el `notifyDataSetChanged()` de `actualizarLista` y entendé el contrato — con 8 ítems no vas a ver la diferencia; en las listas del TP, sí.
2. **Favoritas primero.** Ordená la lista con las favoritas arriba. Vas a descubrir que el orden es *otro* estado más que combinar dentro de `aplicarFiltros()` — y que interactúa con el modo. La función central crece; imaginala sin función central.
3. **El rescate manual.** Arreglá el modo que se apaga *a la manera tradicional*: guardá `soloFavoritas` en `onSaveInstanceState` y restauralo en `onCreate` (el mecanismo del desafío del Laboratorio 2). Funciona — y es exactamente el tipo de código, un rescate a mano por cada variable de cada pantalla, que la parte 3B viene a jubilar.

Checklist de salida

- ☐ AppTeca completa: mis 8 apps, detalle por toque, estrella en lista y en detalle, buscador y modo "solo favoritas" combinados
- ☐ Vi la lista desactualizada tras marcar favorita en el detalle — y sé explicar por qué pasó
- ☐ Vi el modo "solo favoritas" apagarse solo con el giro — y sé explicar por qué
- ☐ Ejecuté la muerte de proceso y sé qué estado murió en cada experimento y por qué
- ☐ Columna A de mi ficha comparativa, completa
- ☐ (Opcional) DiffUtil · favoritas primero · el rescate manual

Adónde va esto: guardá el proyecto tal como quedó — **la parte 3B no arranca de cero: refactoriza exactamente este código.** El estado va a mudarse de la Activity a un lugar que sobrevive; el sincronizador (vos) va a ser reemplazado por una suscripción; y el giro fatal va a dejar de serlo. Mismo producto, otra arquitectura — y tu ficha va a registrar la diferencia.